

23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

3. Database Design Document

Group No : D13

Team Members :

Roll No	Student Name
AM.SC.U4CSE25346	S.V KARTHIK
AM.SC.U4CSE25335	N.RAM LOKESH
AM.SC.U4CSE25340	P. SPUTHNIK
AM.SC.U4CSE25303	ANDE ROOP SRI SAGAR

Project Title :Corporate Compliance and Certification Tracker

3.1. Detailed Description of the Project

System Context and Domain Overview

Organizations must maintain strict compliance standards by ensuring employees complete mandatory training and maintain valid certifications. Tracking these credentials manually causes operational friction, missed renewals, and compliance penalties. The Corporate Compliance and Certification Tracker centralizes workforce credentials, automatically monitors expiry dates, enforces role-based authorization, and logically retains historical records for company audits.

Certification Lifecycle & Workflow

To understand the system's architecture, the standard data workflow follows these sequential steps:

1. Onboarding

An employee joins the company and is securely assigned to a specific department.

2. Certification Earned

The employee successfully completes a training program (e.g., AWS Machine Learning).

3. Data Entry

HR manually records this certification instance into the system using simple data forms, providing its specific issue and expiry dates.

4. Monitoring

The system automatically computes the compliance status based on the current date.

5. Alerting

As the expiry date approaches (e.g., 30 days prior), a proactive notification reminder is generated for the HR team.

6. Renewal & History

When the employee renews the certificate, a brand new certification record is created in the database. The old record is retained unaltered to safely archive a full historical audit trail.

7. Reporting

Managers and HR query the system to view real-time compliance dashboards and generate clean summary reports.

User Roles and Specific Permissions

The system implements strict Role-Based Access Control (RBAC) to ensure everyone only sees the tools and data they need:

Administrator

Possesses full CRUD privileges for system-wide master data. Actions include adding/updating departments, managing operational locations, adding/updating certification providers and programs, managing user accounts, and viewing system-wide audit logs.

HR Staff / Compliance Officer

Oversees corporate compliance. Actions include adding/updating employee records, assigning earned certifications, processing renewals, viewing automated expiry alerts, and generating company-wide audit reports.

Department Manager

Possesses restricted, read-only access. Actions are limited to viewing the certification statuses and compliance summaries strictly for employees assigned to their specific department.

Entities and Attributes

LOCATION

- **Location_ID (Primary Key):** Unique identifier for an operational site.

- **Location_Name:** Name of the site (e.g., HITEC City Campus, Gachibowli Phase 2, Cyberabad GCC).
- **Region:** Geographic region.

DEPARTMENT

- **Department_ID (Primary Key):** Unique code identifying the department.
- **Department_Name:** Name of the division (e.g., AI & Machine Learning, Data Engineering).
- **Location_ID (Foreign Key):** References LOCATION(Location_ID).

EMPLOYEE

- **Employee_ID (Primary Key):** Unique employee identifier (e.g., EMP-1042).
- **First_Name, Last_Name:** Employee's full name.
- **Email:** Unique business email address.
- **Job_Role:** Assigned corporate designation.
- **Department_ID (Foreign Key):** References DEPARTMENT(Department_ID).

PROVIDER

- **Provider_ID (Primary Key):** Unique identifier for external issuing bodies (e.g., AWS, Cisco).
- **Provider_Name:** Name of the certifying vendor.
- **Contact_Email:** Official vendor support contact.

CERTIFICATION_PROGRAM

- **Certification_ID (Primary Key):** Unique program identifier.
- **Title:** Name of the certification.
- **Description:** Scope of the program.
- **Validity_Months:** Standard validity period in months.
- **Provider_ID (Foreign Key):** References PROVIDER(Provider_ID).

EMPLOYEE_CERTIFICATION

Junction / Bridge Entity

- **Record_ID (Primary Key):** Unique tracking identifier for a specific issued certificate.

- **Employee_ID (Foreign Key):** References EMPLOYEE(Employee_ID).
- **Certification_ID (Foreign Key):** References CERTIFICATION_PROGRAM(Certification_ID).
- **Issue_Date:** Date the certificate was awarded.
- **Expiry_Date:** Date validity ends.
- **Compliance_Status (Derived Attribute):** Dynamically computed based on current date vs. Expiry_Date.

USER_ACCOUNT

- **User_ID (Primary Key):** Unique account identifier.
- **Employee_ID (Foreign Key, Candidate Key):** References EMPLOYEE(Employee_ID).
- **Username:** Unique login username.
- **Password_Hash:** Encrypted credential.
- **Role:** System permission level (Admin, HR_Staff, Manager).

NOTIFICATION_REMINDER

- **Notification_ID (Primary Key):** Unique alert identifier.
- **Record_ID (Foreign Key):** References EMPLOYEE_CERTIFICATION(Record_ID).
- **Scheduled_Date:** Date when the alert should trigger.
- **Status:** Tracking state (Pending, Sent, Acknowledged).

SYSTEM_AUDIT_LOG

Expanded for System-Wide Tracking

- **Audit_ID (Primary Key):** Unique log entry identifier.
- **Entity_Type:** The table modified (e.g., 'EMPLOYEE', 'CERTIFICATION_PROGRAM').
- **Entity_ID:** The specific Primary Key of the modified record.
- **Action_Type:** Operation performed (INSERT, UPDATE, DELETE).
- **Action_Timestamp:** Exact date and time of modification.
- **Performed_By:** References USER_ACCOUNT(User_ID).

Relationships, Participation, and Structural Constraints

LOCATION to DEPARTMENT (1:N)

One location can host multiple departments (Partial participation for Location, as a newly added location might not have departments assigned yet). Every department must operate in exactly one location (Total participation for Department).

DEPARTMENT to EMPLOYEE (1:N)

One department employs many employees (Partial participation for Department). Every employee must belong to exactly one department (Total participation for Employee).

PROVIDER to CERTIFICATION_PROGRAM (1:N)

A provider issues multiple certification programs (Partial participation for Provider). Every certification program must be linked to exactly one provider (Total participation for Certification Program).

EMPLOYEE to CERTIFICATION_PROGRAM (M:N Resolution)

An employee can earn multiple certifications, and a certification can be earned by many employees. This is resolved via the EMPLOYEE_CERTIFICATION junction entity:

- **EMPLOYEE to EMPLOYEE_CERTIFICATION (1:N):** An employee can hold multiple certification instances (Partial participation for Employee, as new hires might have zero). Every certification instance MUST belong to exactly one employee (Total participation for Employee_Certification).
- **CERTIFICATION_PROGRAM to EMPLOYEE_CERTIFICATION (1:N):** A program can be issued to many employees (Partial participation for Program). Every instance MUST reference exactly one program (Total participation for Employee_Certification).

USER_ACCOUNT to SYSTEM_AUDIT_LOG (1:N)

Every audit log entry must be associated with exactly one user account through Performed_By. A user account can be associated with multiple audit log entries.

EMPLOYEE to USER_ACCOUNT (1:1)

Not every employee requires system login access (Partial participation for Employee). Every User Account created must belong to exactly one valid Employee (Total participation for User Account).

EMPLOYEE_CERTIFICATION to NOTIFICATION_REMINDER (1:N)

A certification record can generate multiple reminders (Partial participation for Employee_Certification, as some might not have pending alerts). Every reminder must

link to exactly one certification record (Total participation for Notification_Reminder).

Business Rules

1. Uniqueness: Every entity must have a strictly unique, non-null Primary Key. Email in EMPLOYEE and Username in USER_ACCOUNT must also be unique.

2. Assignment: Every employee must be assigned to exactly one active department.

3. Provider Mandate: Every certification program must be tied to exactly one registered provider.

4. Temporal Logic: A certification's Expiry_Date must be chronologically strictly greater than its Issue_Date.

5. Historical Preservation (Renewal Rule): Certification renewal creates a new instance inside EMPLOYEE_CERTIFICATION. Old records are never overwritten, ensuring a complete historical audit trail.

6. Notification Binding: A notification reminder must be associated with a specific, existing certification record.

7. Access Control: Only authenticated users with Admin or HR_Staff roles can modify (INSERT/UPDATE/DELETE) certification records.

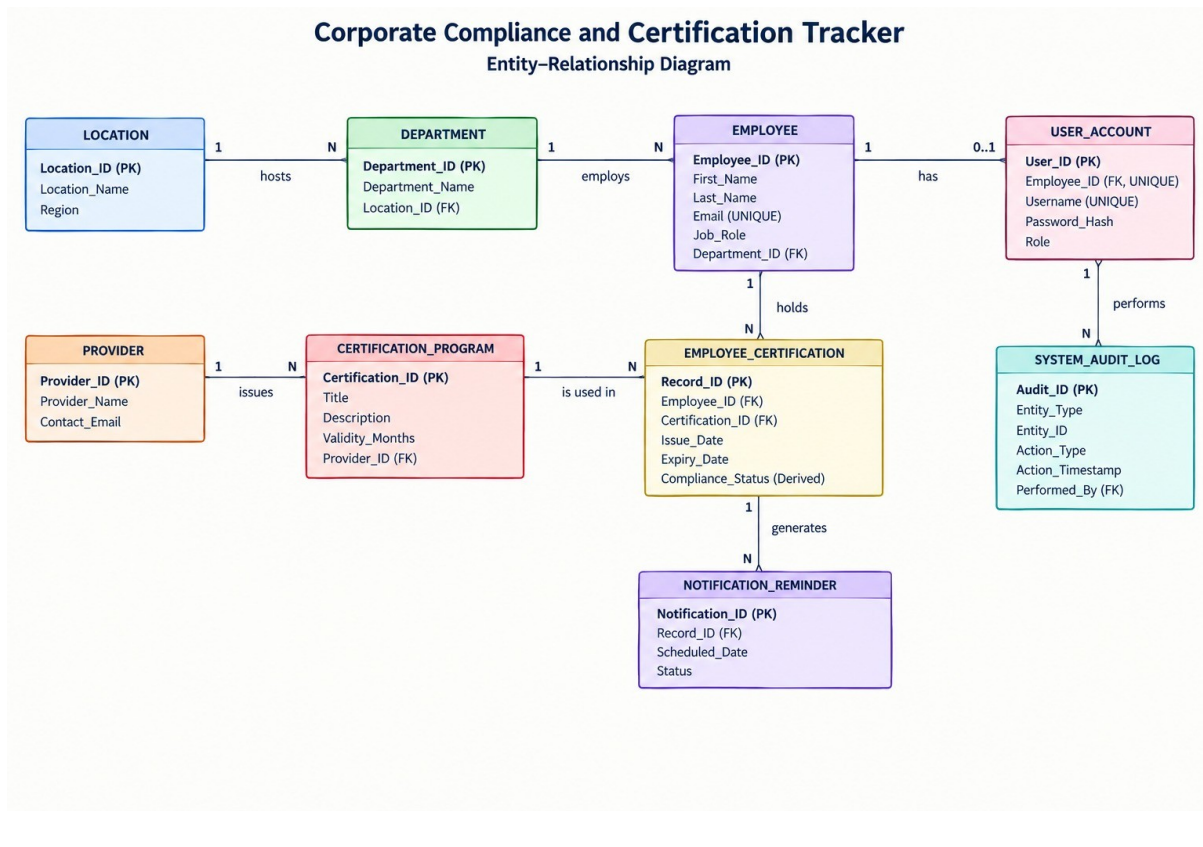
8. Account Mapping: A system user account must be mapped to an existing employee.

9. Audit Immutability: Records within the SYSTEM_AUDIT_LOG are append-only and cannot be updated or deleted by any user, including Administrators.

10. Automated Compliance Status (Derived Logic):

- If $\text{Current_Date} > \text{Expiry_Date} \Rightarrow \text{Compliance_Status} = \text{'Expired'}$
- If $\text{Current_Date} \leq \text{Expiry_Date}$ AND $(\text{Expiry_Date} - \text{Current_Date}) \leq 30 \text{ Days} \Rightarrow \text{Compliance_Status} = \text{'Expiring Soon'}$
- If $(\text{Expiry_Date} - \text{Current_Date}) > 30 \text{ Days} \Rightarrow \text{Compliance_Status} = \text{'Valid'}$

3.2 ER Diagrams



3.3 Relational Schema

LOCATION

Attribute	Constraint
Location_ID	PK
Location_Name	—
Region	—

DEPARTMENT

Attribute	Constraint
Department_ID	PK

Attribute	Constraint
Department_Name	—
Location_ID	FK → LOCATION(Location_ID)

EMPLOYEE

Attribute	Constraint
Employee_ID	PK
First_Name	—
Last_Name	—
Email	U
Job_Role	—
Department_ID	FK → DEPARTMENT(Department_ID)

USER_ACCOUNT

Attribute	Constraint
User_ID	PK
Employee_ID	FK → EMPLOYEE(Employee_ID), U
Username	U
Password_Hash	—
Role	—

PROVIDER

Attribute	Constraint
Provider_ID	PK
Provider_Name	—
Contact_Email	—

CERTIFICATION_PROGRAM

Attribute	Constraint
Certification_ID	PK
Title	—

Attribute	Constraint
Description	—
Validity_Months	—
Provider_ID	FK → PROVIDER(Provider_ID)

EMPLOYEE_CERTIFICATION

Attribute	Constraint
Record_ID	PK
Employee_ID	FK → EMPLOYEE(Employee_ID)
Certification_ID	FK → CERTIFICATION_PROGRAM(Certification_ID)
Issue_Date	—
Expiry_Date	—
Compliance_Status	Derived

NOTIFICATION_REMINDER

Attribute	Constraint
Notification_ID	PK
Record_ID	FK → EMPLOYEE_CERTIFICATION(Record_ID)
Scheduled_Date	—
Status	—

SYSTEM_AUDIT_LOG

Attribute	Constraint
Audit_ID	PK
Entity_Type	—
Entity_ID	—
Action_Type	—
Action_Timestamp	—
Performed_By	FK → USER_ACCOUNT(User_ID)

Notation

PK — Primary Key

FK — Foreign Key

U — Unique

Derived — Attribute whose value is calculated dynamically

3.4 Normalization and Functional dependency analysis

Normalization helps organize the database to reduce redundancy and avoid insertion, deletion, and modification problems. In this section, we identify the functional dependencies and candidate keys using attribute closure, and then analyze the relations from **1NF to BCNF**.

Functional Dependencies Identified in Our Tables

Below we have listed the main functional dependencies (FDs) for each table in our schema, along with the candidate key(s) for that table.

LOCATION

Functional Dependency:

Location_ID $\rightarrow$ Location_Name, Region

Candidate Key: {Location_ID}

DEPARTMENT

Functional Dependency:

Department_ID $\rightarrow$ Department_Name, Location_ID

Candidate Key: {Department_ID}

EMPLOYEE

Functional Dependencies:

Employee_ID $\rightarrow$ First_Name, Last_Name, Email, Job_Role, Department_ID

Email $\rightarrow$ Employee_ID, First_Name, Last_Name, Job_Role, Department_ID

Candidate Keys: {Employee_ID}, {Email}

USER_ACCOUNT

Functional Dependencies:

User_ID $\rightarrow$ Employee_ID, Username, Password_Hash, Role

Employee_ID $\rightarrow$ User_ID, Username, Password_Hash, Role

Username $\rightarrow$ User_ID, Employee_ID, Password_Hash, Role

Candidate Keys: {User_ID}, {Employee_ID}, {Username}

PROVIDER

Functional Dependency:

Provider_ID $\rightarrow$ Provider_Name, Contact_Email

Candidate Key: {Provider_ID}

CERTIFICATION_PROGRAM

Functional Dependency:

Certification_ID $\rightarrow$ Title, Description, Validity_Months, Provider_ID

Candidate Key: {Certification_ID}

EMPLOYEE_CERTIFICATION

Functional Dependency:

Record_ID $\rightarrow$ Employee_ID, Certification_ID, Issue_Date, Expiry_Date, Compliance_Status

Candidate Key: {Record_ID}

NOTIFICATION_REMINDER

Functional Dependency:

Notification_ID $\rightarrow$ Record_ID, Scheduled_Date, Status

Candidate Key: {Notification_ID}

SYSTEM_AUDIT_LOG

Functional Dependency:

Audit_ID $\rightarrow$ Entity_Type, Entity_ID, Action_Type, Action_Timestamp,
Performed_By

Candidate Key: {Audit_ID}

Checking Candidate Keys Using Attribute Closure

To justify our candidate keys properly, we calculated the attribute closure for a few important tables:

1) EMPLOYEE Table – Checking if {Employee_ID} is a Candidate Key

Step 1:

$X^+ = \{\text{Employee_ID}\}$

Step 2: Using Employee_ID $\rightarrow$ First_Name, Last_Name, Email, Job_Role,
Department_ID

$X^+ = \{\text{Employee_ID}, \text{First_Name}, \text{Last_Name}, \text{Email}, \text{Job_Role}, \text{Department_ID}\}$

Since X^+ now contains all the attributes of the EMPLOYEE table, Employee_ID is a candidate key.

Similarly, Email is also a candidate key because Email is a unique attribute in the EMPLOYEE table.

2) USER_ACCOUNT Table – Checking if {User_ID} is a Candidate Key

Step 1:

$X^+ = \{\text{User_ID}\}$

Step 2: Using User_ID $\rightarrow$ Employee_ID, Username, Password_Hash, Role

$X^+ = \{\text{User_ID}, \text{Employee_ID}, \text{Username}, \text{Password_Hash}, \text{Role}\}$

Since X^+ contains all the attributes of the USER_ACCOUNT table, User_ID is a candidate key.

Since Employee_ID and Username are also unique in the table, both can also act as candidate keys.

3) EMPLOYEE_CERTIFICATION Table – Checking if {Record_ID} is a Candidate Key

Step 1:

$X^+ = \{\text{Record_ID}\}$

Step 2: Using $\text{Record_ID} \rightarrow \text{Employee_ID}, \text{Certification_ID}, \text{Issue_Date}, \text{Expiry_Date}, \text{Compliance_Status}$

$X^+ = \{\text{Record_ID}, \text{Employee_ID}, \text{Certification_ID}, \text{Issue_Date}, \text{Expiry_Date}, \text{Compliance_Status}\}$

Since X^+ contains all the attributes of the EMPLOYEE_CERTIFICATION table, Record_ID is a candidate key.

Normalization Process – 1NF to BCNF

1NF (First Normal Form)

All our tables store only single and atomic values in each column. There are no repeating groups or comma-separated values in our schema. Each employee, certification program, certification record, notification, and audit record is stored separately. Therefore, all our tables satisfy 1NF.

2NF (Second Normal Form)

2NF problems occur when a table has a composite primary key and some non-key attribute depends only on part of that key. In our schema, all tables use single-attribute primary keys such as Location_ID, Department_ID, Employee_ID, User_ID, Provider_ID, Certification_ID, Record_ID, Notification_ID, and Audit_ID. Therefore, there are no partial dependencies and all our tables satisfy 2NF.

3NF (Third Normal Form)

3NF requires that non-key attributes should not depend on other non-key attributes. From the functional dependencies identified above, the non-key attributes in each table depend directly on the candidate key of that table. There are no unwanted transitive dependencies between non-key attributes.

The Compliance_Status attribute in the EMPLOYEE_CERTIFICATION table is a derived attribute, calculated based on the certification's expiry information. Since it is derived from the record information and does not introduce a separate dependency between non-key attributes, it does not create a normalization problem in our design.

Therefore, all our tables satisfy 3NF.

BCNF (Boyce-Codd Normal Form)

For BCNF, every determinant in a functional dependency must be a candidate key. Looking at the functional dependencies listed above, the determinant of every non-trivial functional dependency is a candidate key of its respective table.

For example, in the USER_ACCOUNT table, User_ID, Employee_ID, and Username are candidate keys because they uniquely identify the remaining attributes. Similarly, Employee_ID and Email are candidate keys in the EMPLOYEE table.

Therefore, there is no functional dependency where a non-key attribute determines another attribute. Hence, all our tables satisfy BCNF.

Conclusion

After analyzing the functional dependencies, candidate keys, attribute closures, and normalization from 1NF to BCNF, we conclude that our Corporate Compliance and Certification Tracker database is already normalized up to BCNF and does not require any further decomposition. The use of unique primary keys and appropriate foreign keys helps avoid partial and transitive dependency problems in our database design.